

Java Backend Architecture

Interview Series

Chapter 17.1

Kubernetes Architecture

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 17.1 - Kubernetes Architecture

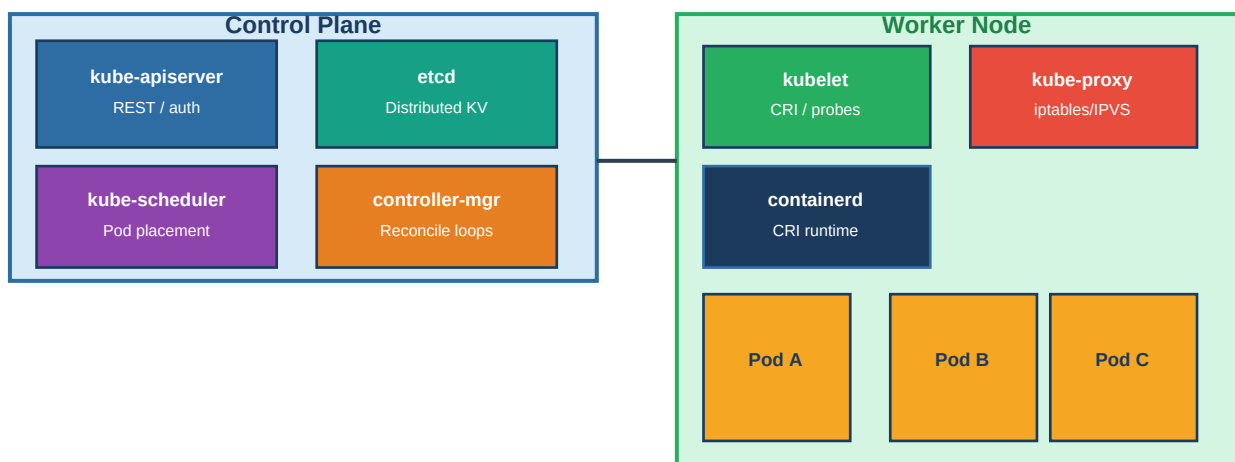
Overview

Kubernetes (K8s) is a production-grade container orchestration platform. It consists of a Control Plane that manages cluster state and Worker Nodes that run workloads. Understanding the architecture is essential for designing resilient, scalable Java backend systems deployed on Kubernetes.

Key Definitions

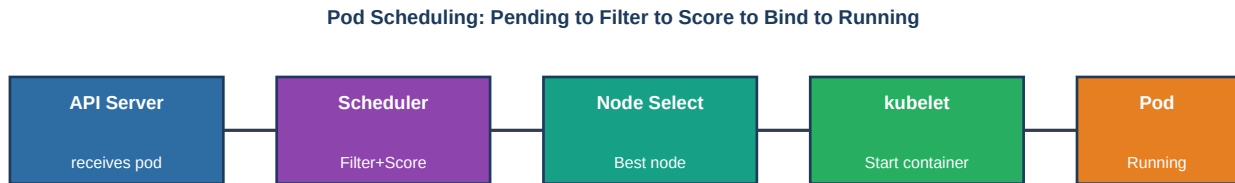
kube-apiserver	Central REST API gateway; handles auth, authz, admission controllers, persists to etcd.
etcd	Distributed key-value store using Raft consensus; holds all cluster state.
kube-scheduler	Assigns pods to nodes: filter unsuitable nodes, score remaining, bind best node.
kube-controller-manager	Runs reconcile loops: Deployment, ReplicaSet, Node, EndpointSlice controllers.
kubelet	Node agent ensuring containers run per PodSpec; manages liveness/readiness/startup probes.
kube-proxy	Maintains iptables/IPVS rules on each node for Service VIP traffic distribution.
containerd	Default CRI-compatible container runtime managing image pulls and container lifecycle.
CNI Plugin	Calico/Cilium/Flannel: configures pod networking, overlays, and NetworkPolicies.
CSI Driver	ebs.csi.aws.com, pd.csi.storage.gke.io: dynamic persistent volume provisioning.
RBAC	Role-Based Access Control: Roles/ClusterRoles bound to ServiceAccounts via RoleBindings.

Diagram 1: Full K8s Architecture



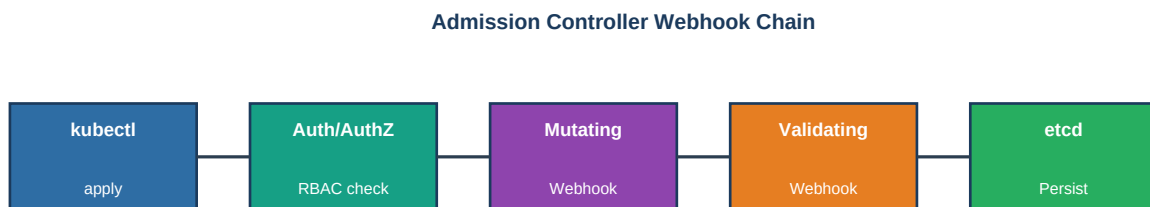
CNI: Calico/Cilium/Flannel | CSI: EBS/PD drivers | RBAC controls access

Diagram 2: Pod Scheduling Flow



Scoring: resource fit, affinity, taints/tolerations, topology spread

Diagram 3: Admission Controller Webhook Chain



Mutating: inject sidecars, set defaults | Validating: enforce policy, reject

Control Plane Component Comparison

Component	Role	Failure Impact	HA Strategy
kube-apiserver	REST gateway, auth, admission	No cluster ops possible	Multiple replicas behind LB
etcd	Cluster state store	Cluster state frozen	Odd-number quorum (3 or 5)
kube-scheduler	Pod placement	New pods stay Pending	Leader election, 2+ replicas
controller-manager	Reconcile loops	Deployments not healed	Leader election, 2+ replicas

Code Examples

Check control plane health:

```
kubectl get componentstatuses kubectl get pods -n kube-system
```

RBAC Role and RoleBinding:

```
apiVersion: rbac.authorization.k8s.io/v1 kind: Role metadata: namespace: default name: pod-reader rules: - apiGroups: ["" ] resources: ["pods"] verbs: ["get", "watch", "list"] --- kind: RoleBinding apiVersion: rbac.authorization.k8s.io/v1 metadata: name: read-pods namespace: default subjects: - kind: ServiceAccount name: my-service-account namespace: default roleRef: kind: Role name: pod-reader apiGroup: rbac.authorization.k8s.io
```

Mutating Admission Webhook:

```
apiVersion: admissionregistration.k8s.io/v1 kind: MutatingWebhookConfiguration metadata: name:
sidecar-injector webhooks: - name: inject.example.com rules: - operations: ["CREATE"]
apiGroups: [""] resources: ["pods"] clientConfig: service: name: sidecar-injector-svc
namespace: default path: "/mutate" admissionReviewVersions: ["v1"] sideEffects: None
```

Interview Q&A; - Kubernetes Architecture

Q1. What is the role of kube-apiserver?

kube-apiserver is the central REST gateway for all cluster operations. It validates and processes API requests, enforces authentication (certificates, bearer tokens, OIDC) and authorization (RBAC), runs admission controllers (mutating then validating webhooks), and persists final state to etcd. All components communicate exclusively through the API server.

Q2. How does etcd store cluster state and what happens if it becomes unavailable?

etcd is a distributed key-value store using Raft consensus. All K8s objects are stored as serialized protobuf under /registry/* paths. If etcd is unavailable: no new scheduling or config changes are possible, but existing pods keep running (kubelet is independent). For HA, run an odd-number quorum (3 or 5 members).

Q3. Describe the pod scheduling algorithm.

Two phases: Filtering removes nodes lacking resources, with taint without toleration, or affinity mismatch. Scoring ranks remaining nodes using plugins: LeastAllocated, NodeAffinity weight, InterPodAffinity, ImageLocality. Highest score wins; ties broken by round-robin. Scheduler calls API server to bind pod to chosen node.

Q4. What does kubelet do and how does it verify container health?

kubelet is the node agent watching API server for PodSpecs assigned to its node and driving containerd via CRI. livenessProbe: restart container if fails. readinessProbe: remove from Service endpoints if fails. startupProbe: disables liveness/readiness until app starts - critical for slow-starting Java apps.

Q5. Explain kube-proxy modes: iptables vs IPVS.

iptables mode: writes DNAT rules to forward Service VIP traffic to random pod IP. O(n) rule evaluation. IPVS mode: uses Linux kernel IPVS hash tables for O(1) lookup with richer load-balancing algorithms (round-robin, least connections, source IP hash). IPVS preferred for clusters with thousands of Services.

Q6. What are CNI plugins and how do Calico/Cilium/Flannel differ?

CNI plugins implement pod networking. Flannel: simple VXLAN overlay, no NetworkPolicy. Calico: BGP routing (no overlay overhead), full NetworkPolicy, optional eBPF dataplane. Cilium: eBPF-exclusive, L7-aware NetworkPolicies (HTTP path filtering), Hubble observability, mTLS mesh. Cilium preferred for high-throughput Java microservices needing deep network visibility.

Q7. How does the admission controller pipeline work?

After authN/authZ: (1) Mutating webhooks in order - can modify object (inject sidecars, set defaults). (2) Object schema validation. (3) Validating webhooks - can only approve or reject. Built-in controllers: NamespaceLifecycle, LimitRanger, ResourceQuota, PodSecurity. External: OPA/Gatekeeper (policy as code), Kyverno.

Q8. What is the controller-manager and how does the Deployment controller work?

kube-controller-manager runs controllers as goroutines. Deployment controller watches Deployment objects; when desired diverges from actual, creates/updates ReplicaSets. ReplicaSet controller ensures correct pod count. Node controller marks nodes NotReady after heartbeat timeout and evicts pods. EndpointSlice controller keeps EndpointSlices in sync with ready pod IPs.

Q9. How does RBAC work in Kubernetes?

Four objects: Role (namespaced, allowed verbs on resources), ClusterRole (cluster-wide), RoleBinding (grants Role to subjects in namespace), ClusterRoleBinding (grants ClusterRole cluster-wide). Subjects: ServiceAccount (pods), User (human), Group. Java apps should use dedicated ServiceAccounts with

least-privilege Roles - never default service account.

Q10. What is a CSI driver and when do you need one?

CSI (Container Storage Interface) is a standard interface for storage plugins enabling dynamic provisioning from cloud storage (AWS EBS: `ebs.csi.aws.com`, GCE PD: `pd.csi.storage.gke.io`). Needed when StatefulSets (Kafka, Cassandra, PostgreSQL) need persistent block storage with dynamic provisioning, volume snapshots, or volume expansion.

Q11. What happens to running pods if the control plane goes down?

Running pods continue - container runtime and kubelet operate independently. kubelet maintains pod state locally. No new pods can be scheduled, failed pods cannot be replaced. Services still route via existing iptables/IPVS rules but EndpointSlices are not updated. Stateless workloads survive control plane outages temporarily.

Q12. How does Raft consensus work in etcd?

Raft elects one leader; all writes go to leader which replicates to majority before committing. 3-node cluster tolerates 1 failure (quorum=2); 5-node tolerates 2. etcd uses WAL for crash recovery and periodic snapshots. For production K8s, etcd needs SSD storage - high latency causes API server timeouts.

Q13. What is the difference between mutating and validating admission webhooks?

Mutating webhooks run first and can modify the JSON object via JSON Patch (inject sidecar, add default limits). Validating webhooks receive the mutated object and can only approve or reject (enforce naming conventions, block privileged containers). Order: all mutating -> object validation -> all validating.

Q14. How does the scheduler handle pod affinity and anti-affinity?

`requiredDuringSchedulingIgnoredDuringExecution` (hard): pod stays Pending if no matching node. `preferredDuringSchedulingIgnoredDuringExecution` (soft): adds weight to scoring. Pod affinity: co-locate pods (cache near app). Pod anti-affinity: spread for HA. `topologySpreadConstraints` is the modern replacement for fine-grained zone spreading.

Q15. Explain taints and tolerations.

Taints on nodes repel pods: `NoSchedule` (no new pods), `PreferNoSchedule` (soft), `NoExecute` (evicts running). Tolerations in pod spec allow scheduling on tainted nodes. Use cases: dedicated GPU nodes (taint `gpu=true:NoSchedule`), master node isolation (`node-role.kubernetes.io/master:NoSchedule`).

Q16. What are startup probes and why important for Java?

Startup probes disable liveness/readiness until app passes startup check. Critical for Java apps with slow JVM initialization (Spring Boot: 30-60s). Without startup probes, liveness failures during startup cause restart loops. Configure: `failureThreshold * periodSeconds >= maximum startup time`.

Q17. How does the API server handle authentication?

Multiple chained authenticators: client certificate (x509), bearer token (static or OIDC JWT), bootstrap token (node joining), webhook token authentication. First to succeed wins. ServiceAccount tokens auto-mounted in pods. OIDC integration allows corporate SSO (Okta, Keycloak).

Q18. What is a DaemonSet vs Deployment?

DaemonSet: one pod per node - log collectors (Fluentd), monitoring agents (Node Exporter), CNI plugins. Automatically added to new nodes, removed from deleted nodes. Bypasses scheduler for placement. Deployment: fixed replica count distributed across nodes, managed rolling updates.

Q19. Explain ReplicaSet and its relationship to Deployment.

ReplicaSet ensures N identical pods running. Deployment manages ReplicaSets for rolling updates: creates new RS with updated template, scales up while scaling down old RS. Rollback: `kubectl rollout undo deployment/app` scales up previous RS. Never manage ReplicaSets directly.

Q20. How does pod eviction work when a node runs low on resources?

kubelet enforces eviction thresholds: soft (warning + grace period), hard (immediate). Defaults: `memory.available < 100Mi`, `nodefs.available < 10%`. Eviction order: BestEffort (no requests/limits) first, then Burstable, finally Guaranteed (requests==limits). Set requests==limits for Java pods for Guaranteed QoS class.

Q21. What is the resource model in Kubernetes?

Requests: scheduler guarantee - find node with sufficient capacity. Limits: maximum usage; CPU throttling via cgroups, memory OOMKill when exceeded. For Java: set memory request==limit (Guaranteed QoS). Use `-XX:MaxRAMPercentage=75` for heap sizing. CPU request << limit allows bursting during load spikes.

Q22. Describe how kube-proxy implements ClusterIP Services.

Service creation assigns virtual IP from service CIDR. kube-proxy watches Service and EndpointSlice objects. iptables mode: DNAT rules randomly selecting pod IP (probabilistic via statistics module). IPVS mode: virtual server in kernel IPVS table with real servers for each pod endpoint.

Q23. How does Kubernetes handle node failures?

Node controller monitors heartbeats. After node-monitor-grace-period (40s) without heartbeat, node marked NotReady. After pod-eviction-timeout (5 min), pods marked for deletion and Deployment controller creates replacements. PodDisruptionBudget prevents too many pods unavailable during voluntary disruptions.

Q24. What are CustomResourceDefinitions (CRDs)?

CRDs extend the K8s API with custom resource types. Define spec.group, spec.names, spec.versions with OpenAPI v3 schema validation. Once applied, kubectl and API server handle CRUD for the new type. Controllers (operators) watch CRDs and implement business logic - pattern used by cert-manager, Argo CD, Prometheus Operator.

Q25. How does leader election work for controller-manager and scheduler?

Both use K8s Lease objects for leader election. Each replica tries to acquire Lease with renewDeadline; holder renews every leaseDuration. If leader fails to renew, another replica acquires and becomes leader. Ensures exactly one instance reconciling at a time. Same pattern used by Java operators (Fabric8/JOSDK).

Q26. What is containerd and how does it relate to Docker?

containerd is the industry-standard CRI-compatible runtime used by K8s (since 1.24 removed dockershim). Manages image pulls, container lifecycle, namespaces. Docker uses containerd internally. Other CRI runtimes: CRI-O (lightweight, OpenShift), gVisor (sandboxed), Kata Containers (VM-based).

Q27. Explain NetworkPolicy enforcement.

NetworkPolicy selects pods via podSelector and defines allowed ingress/egress by podSelector, namespaceSelector, or ipBlock. By default all pod-to-pod traffic allowed; once any NetworkPolicy selects a pod, only explicitly allowed traffic permitted. Enforced by CNI plugins (Calico, Cilium). Cilium supports L7 policies (HTTP path, method).

Q28. What are PodSecurityAdmission standards?

PSA (replaced PodSecurityPolicy in 1.25+) has three standards: privileged (no restrictions), baseline (prevents privileged containers, hostPID, hostNetwork, dangerous capabilities), restricted (read-only root FS, drops ALL capabilities, runAsNonRoot). Enforce per namespace:

`pod-security.kubernetes.io/enforce=restricted`.

Q29. How do you upgrade a K8s cluster with zero downtime?

Control plane first: drain master, upgrade kubelet/kubeadm, upgrade components, uncordon. Repeat per master. Worker nodes: `kubectl drain` (evicts pods respecting PDB), upgrade kubelet, `kubectl uncordon`. PodDisruptionBudgets prevent all replicas from being evicted simultaneously. Blue-green: provision new cluster, migrate workloads, switch DNS.

Q30. What is the difference between ClusterRole and Role?

Role is namespace-scoped: grants permissions within a specific namespace. ClusterRole is cluster-scoped: grants permissions on cluster-level resources (nodes, PVs) or namespace resources across all namespaces. ClusterRole bound via RoleBinding (not ClusterRoleBinding) grants permissions only in binding namespace - useful for reusable role definitions.

Q31. Explain how resource quotas work.

ResourceQuota limits aggregate consumption per namespace: cpu/memory requests+limits, object counts. LimitRange sets default requests/limits for containers without them and enforces min/max per container. Together prevent one team consuming all cluster resources. Java teams should set LimitRange defaults to ensure all pods have resource requests for proper scheduling.

Q32. What is kubelet container lifecycle management?

Container transitions: Waiting to Running to Terminated. kubelet calls CRI: RunPodSandbox, CreateContainer, StartContainer. postStart lifecycle hook runs after start (async). preStop hook before stop - use for graceful Java shutdown (drain connections, flush buffers). terminationGracePeriodSeconds (default 30s) gives app time to shut down before SIGKILL.

Q33. How does horizontal scaling work?

HPA controller polls metrics-server every 15s, calculates desired replicas = $\text{ceil}(\text{currentReplicas} * \text{currentMetric} / \text{desiredMetric})$, applies min/max bounds, updates Deployment. Scale-up: aggressive (3 min cooldown). Scale-down: conservative (5 min cooldown) to prevent thrashing. Custom metrics via KEDA allow scaling on any Prometheus metric.

Q34. What is the Operator pattern?

Operators encode operational knowledge as code. An Operator is a controller managing a CRD. Watches custom resource and reconciles actual to desired state - handles creation, scaling, backups, upgrades of stateful apps. Examples: Prometheus Operator, Strimzi (Kafka), CockroachDB Operator. Java operators: Fabric8 or JOSDK (Java Operator SDK).

Q35. Explain PodDisruptionBudgets.

PDB limits pods disrupted during voluntary disruptions (node drain, rolling update). minAvailable: 2 ensures at least 2 pods always available. maxUnavailable: 1 allows at most 1 down. During `kubectl drain`, operation waits if eviction would violate budget. Critical for Java microservices maintaining availability during cluster maintenance.

Q36. How does the scheduler handle topology spread constraints?

TopologySpreadConstraints distribute pods across topology domains (zones, nodes). Configure maxSkew (allowed imbalance), topologyKey (e.g. `topology.kubernetes.io/zone`), whenUnsatisfiable (DoNotSchedule or ScheduleAnyway), labelSelector to match pods. maxSkew=1 across zones ensures zone-balanced distribution. Modern replacement for PodAntiAffinity.

Q37. What is the significance of the API server watch mechanism?

Watch (HTTP long-polling or HTTP/2 streaming) is foundation of K8s event-driven architecture. Controllers, scheduler, kubelet use informers (client-side cached watches) to react to object changes. Watches send events: ADDED, MODIFIED, DELETED. Informers maintain local cache reducing API server load. Resource version mechanism ensures no events missed during reconnection.

Q38. How do you troubleshoot a pod stuck in Pending state?

`kubectl describe pod`: Events section shows scheduling failures. Common causes: insufficient resources, node affinity mismatch, taint without toleration, PVC not bound, image pull secret missing. `kubectl get events --sort-by=.metadata.creationTimestamp`. Check node capacity: `kubectl describe nodes | grep -A5 Allocated`.

Q39. What are the possible pod phases?

Pending: accepted but containers not started (waiting for scheduling or image pull). Running: at least one container running. Succeeded: all containers exited 0 (Jobs). Failed: all terminated with at least one non-zero exit. Unknown: node communication lost. Container states: Waiting (ContainerCreating, CrashLoopBackOff), Running, Terminated.

Q40. Explain resourceVersion and generation in K8s objects.

`resourceVersion`: etcd revision number, changes on every write, used by watch to resume. `generation`: increments only when spec changes (not status). `observedGeneration` in status indicates if controller processed latest spec change. `Deployment.status.observedGeneration == metadata.generation` means controller reconciled latest state.

Q41. How does garbage collection work in Kubernetes?

Owner references link dependent objects (Pod -> ReplicaSet -> Deployment). GC controller watches for objects with owner references where owner is deleted. Foreground: owner marked `deletionTimestamp`, dependents deleted, then owner deleted. Background (default): owner deleted immediately, dependents cleaned up asynchronously. Orphan policy: dependents kept when owner deleted.

Q42. What are security implications of default service accounts?

Every pod gets default ServiceAccount with automatically mounted token that can query K8s API. Best practices: `automountServiceAccountToken`: false unless needed, dedicated ServiceAccounts per app, minimal RBAC permissions, use projected volume tokens with audience/expiration instead of long-lived tokens.

Q43. How do node labels and node selectors work?

Node labels: key-value pairs on nodes (`kubectl label node node1 disktype=ssd`). `nodeSelector` in pod spec: simplest form, only schedule on nodes with matching labels. Node affinity: `requiredDuringScheduling` (hard) and `preferredDuringScheduling` (soft) with operators (In, NotIn, Exists, Gt). Place IO-intensive Java apps on SSD nodes.

Q44. What is a headless service?

Headless Service (`clusterIP: None`) has no virtual IP. DNS returns individual pod IPs directly. Used by StatefulSets so each pod (`pod-0.svc.ns.svc.cluster.local`) is individually addressable. Required for stateful clusters (Cassandra, MySQL Group Replication, Kafka) where clients need specific node connections. Also used with custom client-side load balancing.

Q45. Explain container resource limit enforcement.

CPU limits enforced via `cgroups cpu.cfs_quota_us` - throttled when exceeding quota in 100ms period. Memory limits use `cgroup memory.limit_in_bytes` - OOM killer fires when exceeded. For Java: use `-XX:+UseContainerSupport` (JDK 10+ default). `-XX:MaxRAMPercentage=75` sets heap to 75% of container memory limit, leaving room for off-heap (Netty, Metaspace, thread stacks).

Q46. What is a static pod?

Static pods managed directly by kubelet without API server - kubelet reads manifests from /etc/kubernetes/manifests. Used exclusively for control plane components (kube-apiserver, etcd, kube-scheduler, kube-controller-manager) because they must exist before API server is available. Static pods mirrored in API server as read-only mirror pods.

Q47. How does Kubernetes handle secret rotation?

K8s does not natively rotate secrets. Options: External Secrets Operator syncs from Vault/AWS SM with TTL-based refresh. Vault Agent Injector writes secrets to pod filesystem with lease renewal. Manual: update Secret object - pods must restart for env var changes; volume mounts update automatically within ~60s via kubelet sync loop.

Q48. What is the difference between kubectl apply and kubectl create?

kubectl create: imperative, creates resource, fails if exists. kubectl apply: declarative, creates or patches using 3-way merge (desired, last-applied annotation, live). kubectl apply --server-side uses Server-Side Apply where server tracks field managers and handles merge conflicts between multiple actors. Preferred for GitOps.

Q49. How does liveness probe failure trigger restart?

kubelet runs probe at intervalSeconds frequency. After failureThreshold consecutive failures, kubelet marks container unhealthy and restarts per restartPolicy (Always/OnFailure/Never). Container stopped with SIGTERM (then SIGKILL after terminationGracePeriodSeconds). Restart backoff: 10s, 20s, 40s, 80s, 160s, capped at 300s (CrashLoopBackOff).

Q50. What is the cloud-controller-manager?

Runs cloud-specific controllers separately: Node controller (checks if deleted nodes exist in cloud), Route controller (sets up cloud network routes for pod CIDR), Service controller (provisions/updates cloud load balancers for LoadBalancer Services). Allows cloud providers to release controllers independently. On GKE/EKS, runs as managed component.

Q51. Explain the watch/informer pattern used by controllers.

Informers use ListWatch: initial List to populate local cache, then Watch to stream delta events. Events queued in rate-limited work queue. Reconcile loop dequeues items, compares actual vs desired, makes API calls to converge. Controllers are idempotent - resync periodically. SharedIndexInformer shares watches across multiple controllers for the same resource type.

Q52. How do you debug OOMKilled pods?

kubectl describe pod: Last State shows exit code 137 and Reason: OOMKilled. Check memory usage before kill: kubectl top pods. Fix: increase memory limit or fix memory leak. For Java: -XX:+HeapDumpOnOutOfMemoryError, review GC logs (-Xlog:gc*), check off-heap (Netty: -XX:MaxDirectMemorySize, Metaspace: -XX:MaxMetaspaceSize).

Q53. What are finalizers in Kubernetes?

Finalizers in metadata.finalizers prevent object deletion until external cleanup is done. When you delete an object with finalizers, K8s sets deletionTimestamp but does not delete it. Controller performs cleanup (delete cloud resources, flush data) then removes finalizer key. Once all finalizers removed, K8s deletes the object. Used by Operators for cloud resource cleanup.